TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Macroarea di Ingegneria
Dipartimento di Ingegneria Civile e Ingegneria Informatica

Self-adaptive Distributed Systems

Corso di Sistemi Distribuiti e Cloud Computing

A.A. 2025/26

Valeria Cardellini

Laurea Magistrale in Ingegneria Informatica

Self-adaptive software systems

"Intelligence is the ability to adapt to changes"
S. Hawking

- We aim to a software system capable of adapting its operations at run-time with respect to itself and the environment: a self-adaptive (or autonomic) software system
  - Applications of self-adaptive systems in many computing environments
    - Cloud computing
    - Edge/fog computing
    - Compute continuum
    - HPC
    - Cyber-physical systems

Self-adaptive software systems

- **Autonomic computing**: computing paradigm designed to the complexity and heterogeneity of IT systems through automatic adaptations
  - Inspired by human autonomic nervous system, which controls vital functions (heart rate, digestion, body temperature, ...) masking complexity from conscious human control

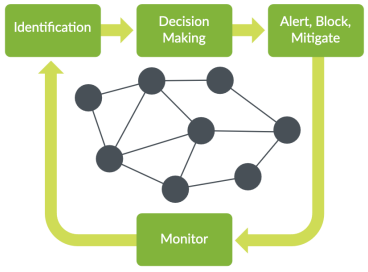
- Key capabilities of self-adaptive (or autonomic) software systems:
  - Autonomously manage their functionalities and goals, with little or no human intervention
  - Adapt to changes and uncertainty in both the environment and the system itself

Kephart and Chess, The vision of Autonomic Computing, *IEEE Computer*, 2003

Self-adaptation is everywhere

kubernetes

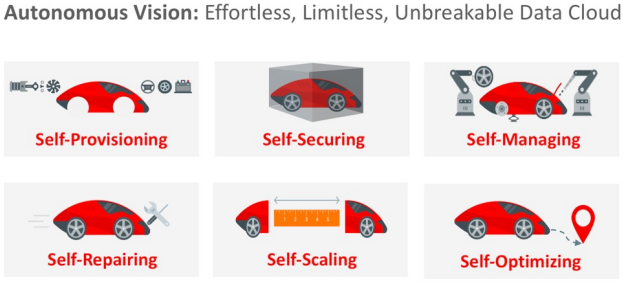
kubernetes.io



Self-healing networks, www.juniper.net



aws.amazon.com/ec2



Autonomous database, www.oracle.com/autonomous-database/what-is-autonomous-database/

Goals of self-adaptive systems

- A self-adaptive (***self-****) software system is able to self-manage, pursuing the following goals:
- Self-optimize
  - Capability of system to optimize its resource usage or performance while providing its required quality goals
  - E.g., change placement of application components onto system nodes to satisfy application response time
- Self-heal
  - Capability of system to discover, diagnose and recover from faults to provide its required quality goals or degrade gracefully otherwise
  - E.g., detect crashed nodes and exclude them from serving requests

Goals of self-adaptive systems

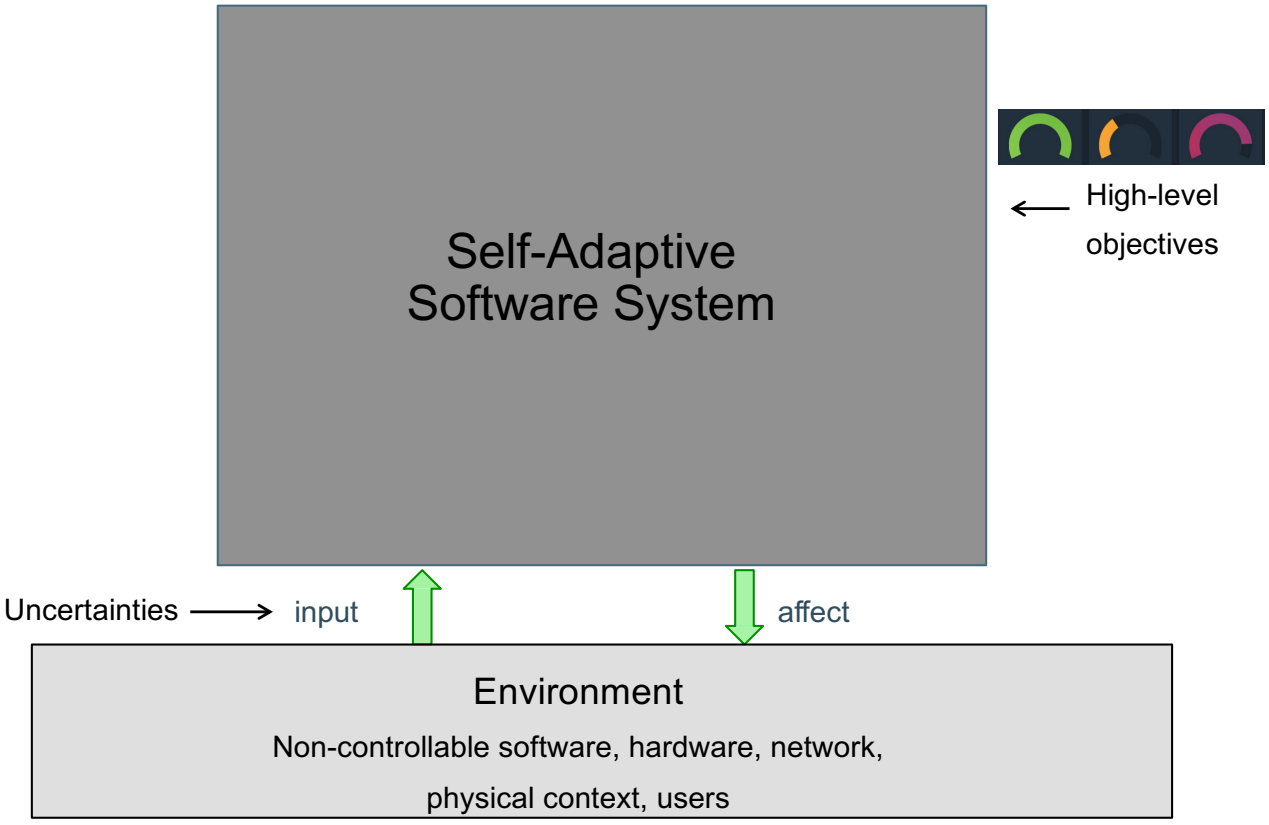
- Self-configure
  - Capability of system to automatically integrate new elements, without interrupting the system's normal operation, or tune some configuration parameters
  - E.g., discover new nodes and add them to serve requests
- Self-protect
  - Capability of system to detect anomalies (i.e., intrusion detection) and react to intrusion and attack actions and its consequences (i.e., intrusion response) so to protect from security threats
  - E.g., in a network of IoT devices detect a jamming attack that corrupts network traffic and adapt packets schedule

How to achieve the goals of a self-* system?

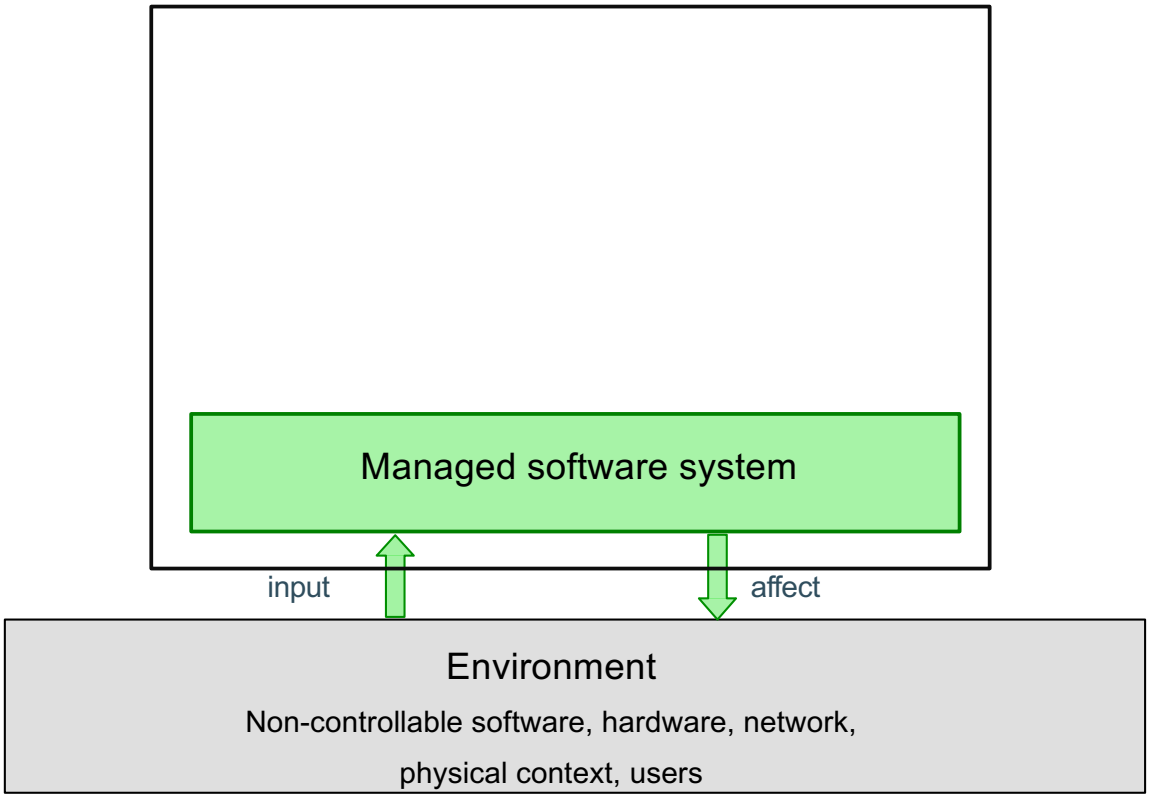
- The system should know its internal state (self-awareness) and the current external operating conditions (self-situation)
- Should identify changes regarding its state and the surrounding environment (self-monitoring)
- And should adapt consequently (self-adjustment)

- These attributes are the implementation mechanisms

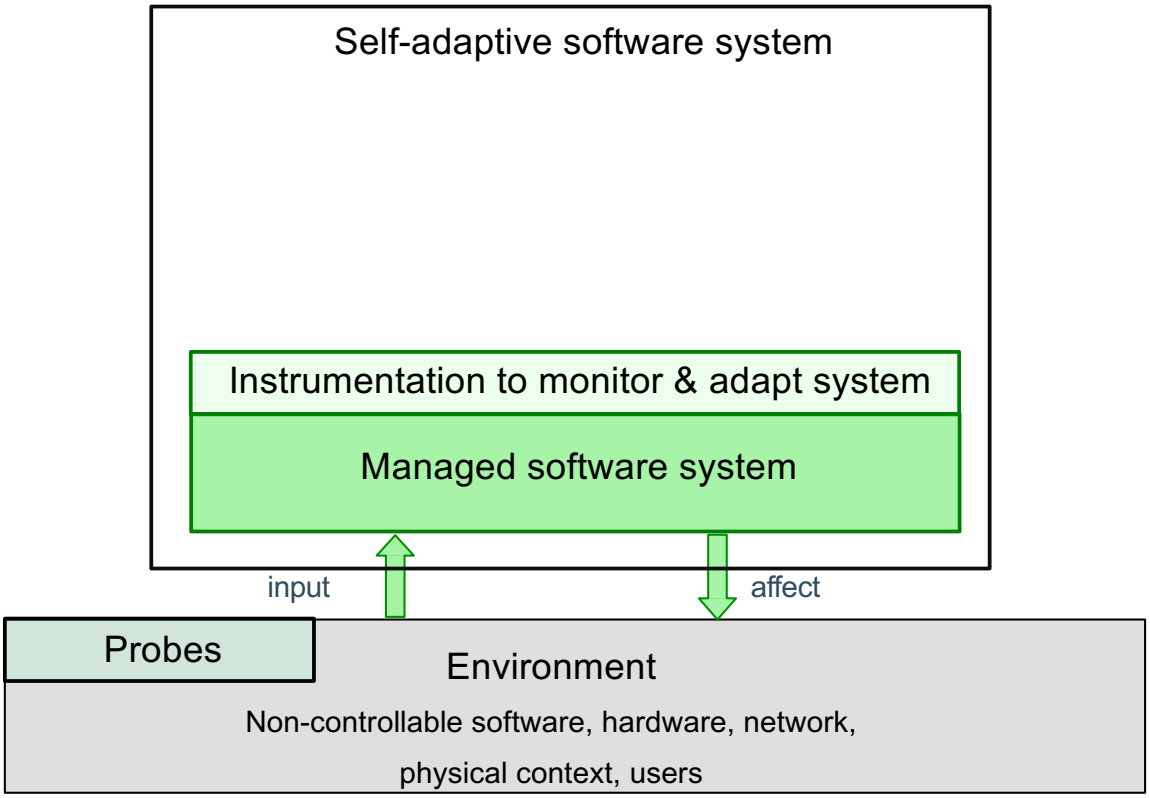
Conceptual model of self-adaptive system



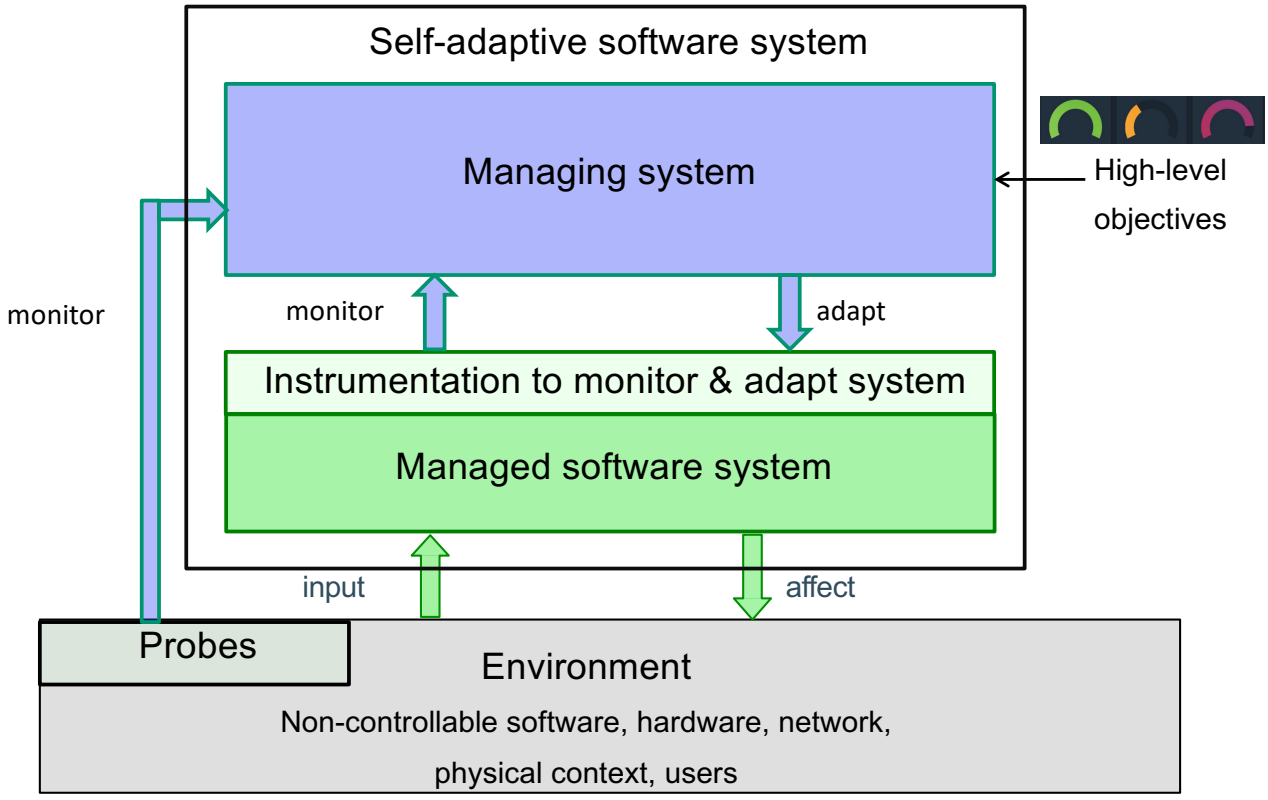
Conceptual model of self-adaptive system



Conceptual model of self-adaptive system

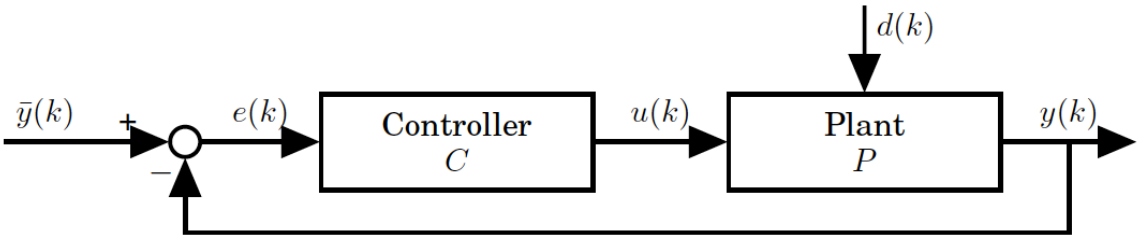


Conceptual model of self-adaptive system



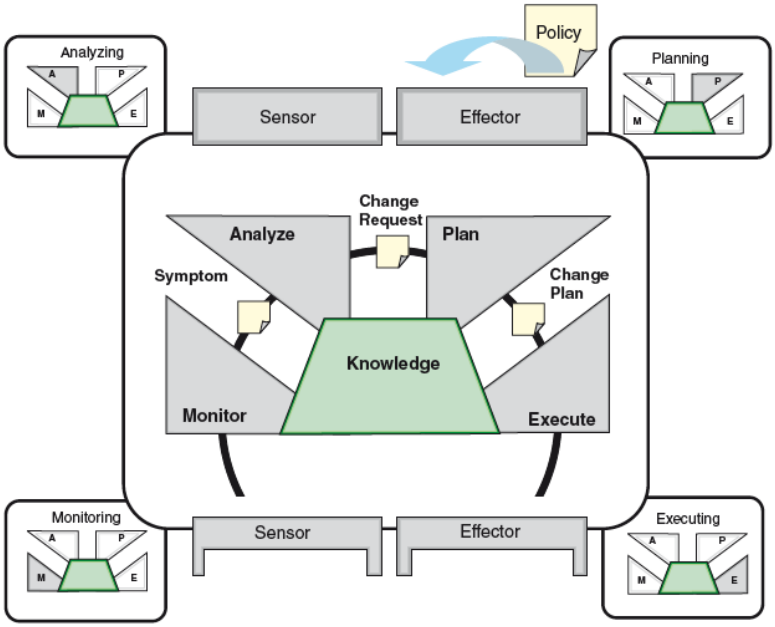
You are already familiar with this model

- The control-theory perspective of a self-adaptive system



MAPE: reference architecture for self-adaptive system

- **MAPE** (Monitor, Analyze, Plan, Execute) loop



MAPE: building blocks (or phases)

- **M**onitor
  - Collects data from the managed system and execution environment through sensors; aggregates, filters and correlates these data into symptoms that can be analyzed
- **A**nalyze
  - Observes and analyzes situations to determine need for adaptation
  - If adaptation is required, it triggers Plan
- **P**lan
  - Determines which mitigation actions need to be performed so to enact a desired alteration in the managed system
- **E**xecute
  - Enacts the change plan by carrying out the actions determined by Plan through effectors so to adapt the managed system
- Plus **K**nowledge (MAPE-K)
  - Stores shared knowledge regarding relevant aspects of the managed system, environment, and the administrator's goals

MAPE: Monitor

- Main design options for Monitor:
  - *When* to monitor: continuously, on demand
  - *What* to monitor: resources, workload, performance, …
  - *How* to monitor: architecture (centralized vs. decentralized), methodology (active vs. passive)
  - *Where* to store monitored data (e.g., time-series database) and *how* (e.g., some pre-processing)

MAPE: Analyze

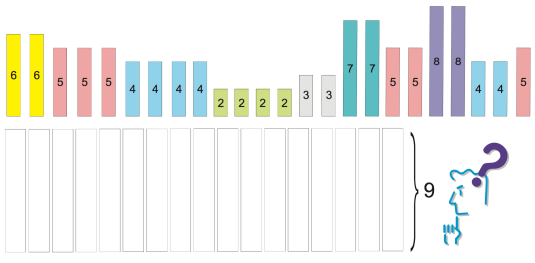
- Main design options for Analyze:
  - *When* to analyze: event- or time-triggered
  - Reactive vs. proactive adaptation
    - *Reactive*: *in reaction to* events that have already occurred (e.g., increase number of resources *after* workload increase)
    - *Proactive*: based on *prediction* so to plan adaptation actions in advance (e.g., increase number of resources *before* workload increase occurs)

MAPE: Plan

- The most challenging and studied MAPE phase
- A variety of methodologies and techniques can be used to plan adaptation, including
  - Optimization theory
    - ✓ Optimal adaptation actions
    - ✗ Con: formulation can be NP-hard, too expensive to solve at runtime
  - Heuristics
    - ✓ Faster
    - ✗ Sub-optimal adaptation actions
  - Machine learning, including reinforcement learning
  - Control theory
  - Queueing theory
- Example: optimal bin packing and heuristic policies to dynamically place virtual machines or containers on servers

Example: VM/container placement

- A set of physical servers (bins) with limited capacity
- How to place a set of virtual machines (VMs) or containers, each with specific resource demands?
- We can use a bin packing optimization problem
- Bin packing: pack items of varying sizes (VM or containers) into the minimum number of bins (servers), each with limited capacity (resources), ensuring the total size in each bin does not exceed its capacity https://en.wikipedia.org/wiki/Bin_packing_problem
- Integer optimization problem ⇒ NP-hard ⇒ we need efficient heuristics to find new placement quickly when some change occurs

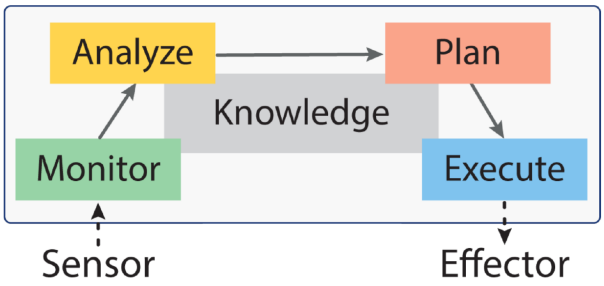


Example: VM/container placement

- Examples of baseline heuristic policies
- *Round robin:* organize bins in a *circular* list, saving the latest used bin; place each item in the next bin with enough capacity, starting from the current position
  - Does not minimize the number of used bins
- *First fit:* organize bins in a list and place each item into *the first bin* in which it fits, starting from the beginning of the list for each item
  - First fit decreasing: variant in which items are sorted in decreasing order
- *Best fit*: place the item in the bin with the *least remaining space* that still fits the item
- *Worst fit*: place the item in the bin with the *most remaining space* where it fits

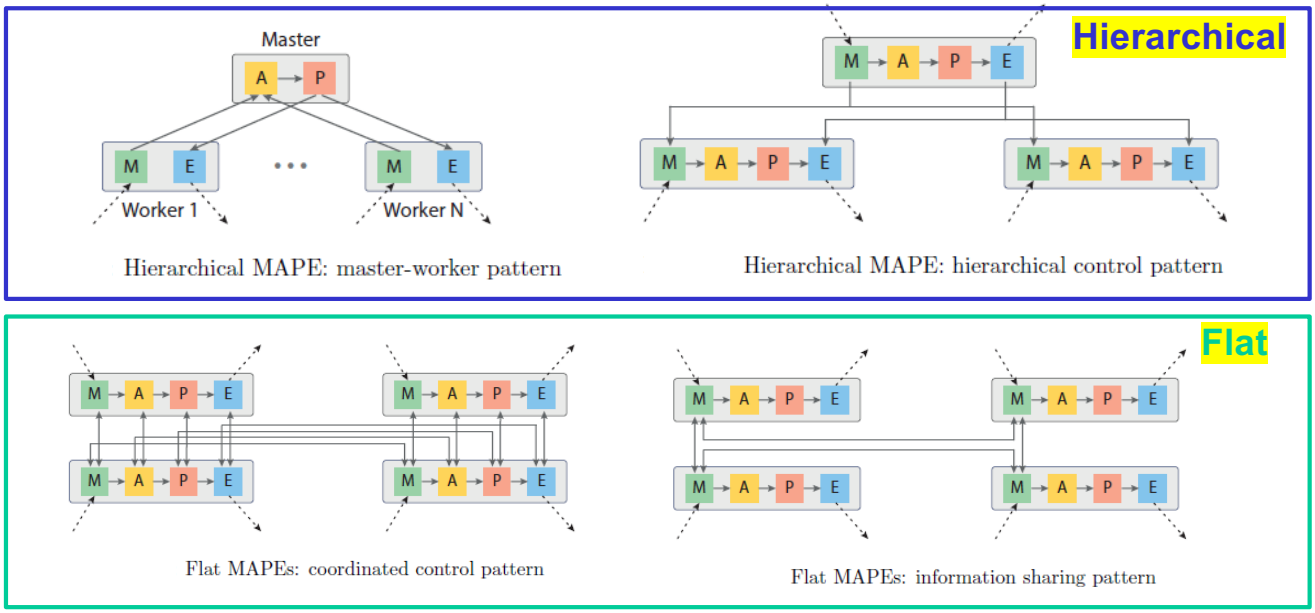
Alternative architectures for MAPE

- Designing the architecture of the managing system
- Centralized MAPE
  - All MAPE components are located on the same node
  - Simple, but lacks of scalability in geo-distributed environments
- **Decentralized MAPE**
  - MAPE components are distributed across multiple nodes
  - Various architectural patterns with its own trade-offs
    - There is no one-size-fits-all solution: the choice depends on system characteristics, application requirements, and operational context



How to decentralize the adaptation control

- Main architectural patterns for decentralized MAPE



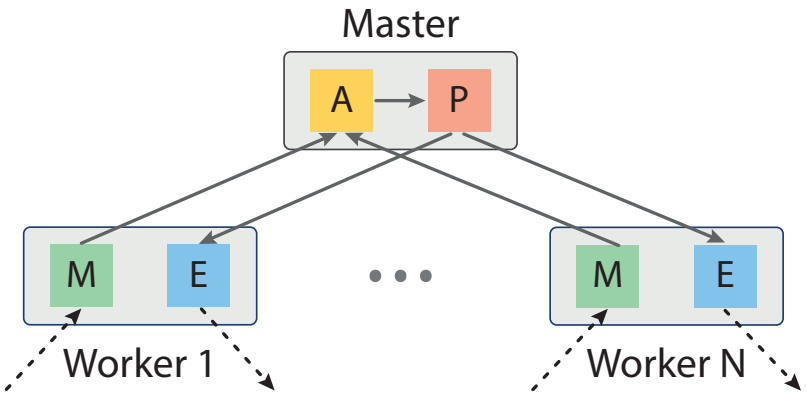
Weyns et al., On patterns for decentralized control in self-adaptive systems. In *SEfSAS II*, 2013

How to decentralize the adaptation control

- First design choice: ***hierarchical*** vs. ***flat***
  - Hierarchical: easier to design, but risk of bottleneck in top level of hierarchy
  - Flat: more difficult to coordinate, but can scale better
- Hierarchical MAPE patterns: multiple MAPE loops organized in a hierarchy, where a higher-level control loop manages subordinated control loops
  - Master-worker
  - Hierarchical control
- Flat MAPE patterns: multiple MAPE loops cooperate as peers
  - Coordinated control
  - Information sharing

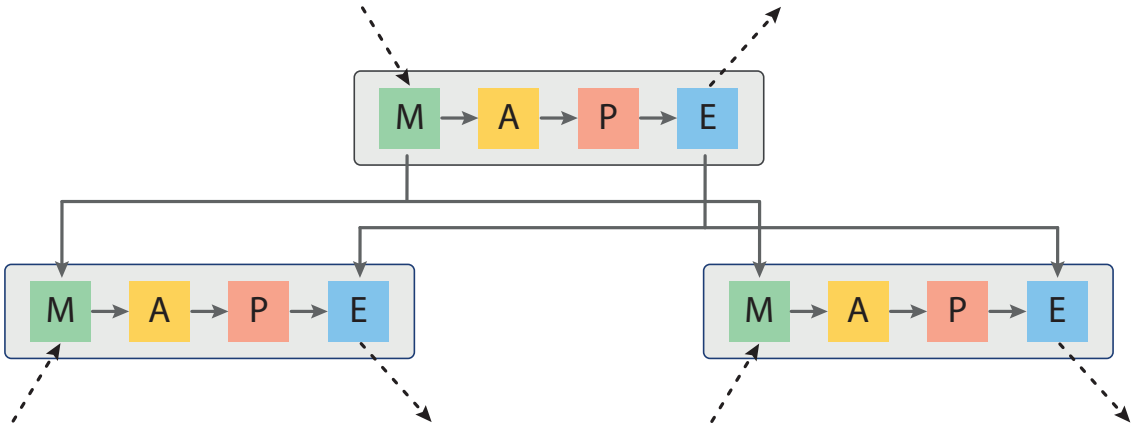
Hierarchical MAPE: master-worker pattern

- Decentralize M and E on workers, keep A and P centralized on master
- ✓ Global view on master who can achieve global adaptation goals
- ✗ Communication overhead and risk of performance bottleneck and SPOF on master



Hierarchical MAPE: hierarchical control pattern

- Multiple MAPE loops, which can operate at different time scales and with separation of concerns
- ✓ Top-level MAPE can achieve global goals, increased flexibility
- ✗ Can be non-trivial to identify different levels of control, depends on managed system characteristics

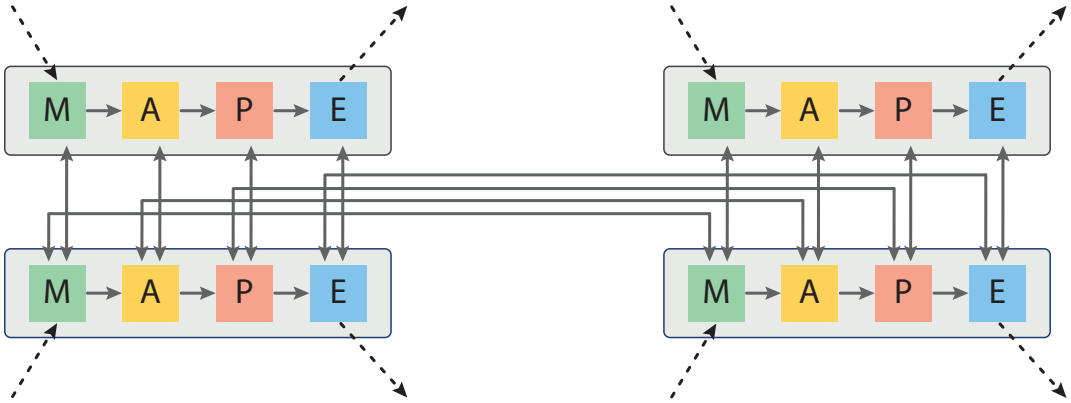


Flat MAPE: coordinated control pattern

- Multiple control loops, each one in charge of some part of the managed system but coordinated through interaction

✓ Better scalability

✗ More difficult to take joint adaptation decisions

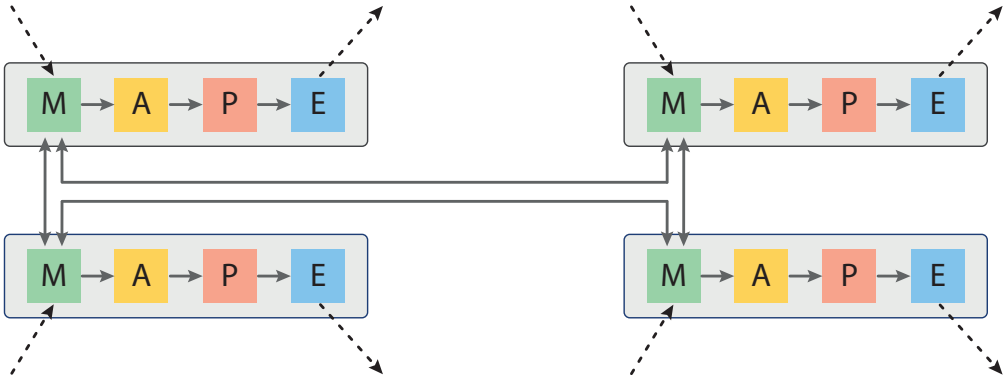


Flat MAPE: information sharing pattern

- Special case of coordinated control pattern: interaction only among M components

✓ Better scalability

✗ Lack of coordination on planning, conflicting or sub-optimal adaptation actions can be enacted

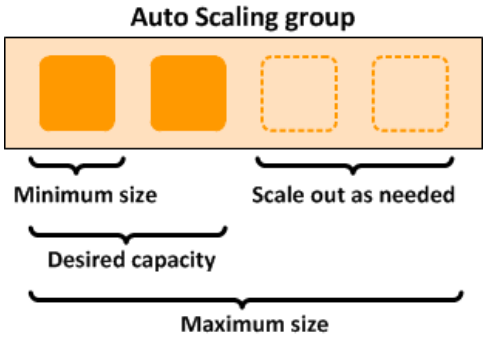


Examples of self-adaptive systems

- Let's analyze 3 examples of self-adaptive systems for resource management
  1. Auto-scaling EC2 instances
  2. Self-adaptive selection of application component services
  3. Auto-scaling microservice-based applications
- Common ground
  - Applications operate under uncertainty (e.g., workload surge and spikes, node crashes)
  - Adaptation goal: maintain target SLO (e.g., application response time, availability, cost)
- Examples vary in planning methodologies and control architectures

Example 1: Amazon EC2 Auto Scaling

- AWS service that automatically adds or removes EC2 instances based on user-defined conditions and health checks https://aws.amazon.com/ec2/autoscaling/



- MAPE Monitor: monitor scaling metrics on EC2 instances using CloudWatch https://aws.amazon.com/cloudwatch/
- Which scaling metrics?
  - CPU utilization, memory usage, network throughput, Application Load Balancer (ALB) request count, …

Example 1: Amazon EC2 Auto Scaling

- MA**P**E Plan: we study 2 policies implemented in AWS Auto Scaling for scale-out/scale-in decisions
  https://docs.aws.amazon.com/autoscaling/ec2/userguide/scale-your-group.html
  - Dynamic simple scaling: responds to real-time metrics (e.g., CPU utilization) with scaling actions triggered by thresholds
  - Predictive scaling: uses machine learning to forecast future traffic and proactively adjusts capacity

- Beyond scaling: self-healing capabilities
  - Auto Scaling is also a self-healing system
  - It detects unhealthy EC2 instances, automatically terminates them, and launches replacements to maintain availability

Example 1: Amazon EC2 Auto Scaling

- **Dynamic simple scaling**: a simple, rule-based policy based on predefined metric thresholds
- Threshold-based heuristic policy (*reactive*)
  - User sets upper and lower thresholds on some scaling metric(s)
  - Upper threshold → triggers scale-out
  - Lower threshold → triggers scale-in

```
if (metric > upper_thr) scale-out
else if (metric < lower_thr) scale-in
```

    - Example of *scale-out* rule: if average CPU utilization of all instances > 70% in last 1 minute, then add 1 new instance
    - Example of *scale-in* rule: if average CPU utilization of all instances is <35% in last 5 min, then remove 1 instance
  - CloudWatch monitors and sends alarms for scaling out (`upper_thr`) and scaling in (`lower_thr`)
  - Cooldown period: delay after a scaling action before any further scaling actions can be triggered

Threshold-based policy: pros & cons

✓ Simple and easy-to-understand
- Choose scaling metric(s), a monitoring period, and upper/lower thresholds for triggering alarms

- Basic idea, but in practice…

✗ Choosing the right metrics and thresholds is difficult
- Metrics are application-dependent: application components may be CPU-, memory-, I/O-intensive, or a mix
- Thresholds can be too aggressive or too conservative; examples of common issues:
  - Slow scale-out: upper threshold too high → not enough instances added → SLO violations
  - Rapid scale-out: upper threshold too low → too many instances added → underutilization and high cost
  - Slow scale-in: scaling period too long → not enough instances removed → wasted resources and cost
  - Rapid scale-in: lower threshold too high → too many instances removed → SLO violations

Threshold-based policy: pros & cons

- Basic idea, but in practice…

✗ No application-specific metrics (e.g., response time often missing)

✗ Not robust against varying load patterns

- Idea: we can exploit reinforcement learning to learn optimal threshold values

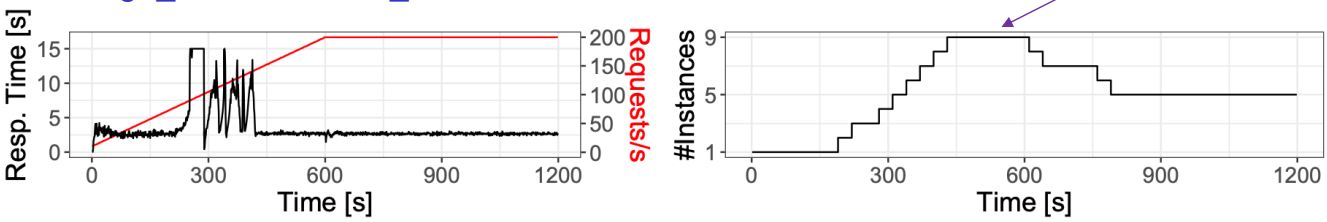
Rossi et al., Dynamic multi-metric thresholds for scaling applications using reinforcement learning, *IEEE Trans. Cloud Comp.*, 2023
https://doi.org/10.1109/TCC.2022.3163357

Threshold-based policy: cons

- Examples of wrong policy configurations
  - CPU utilization as scaling metric

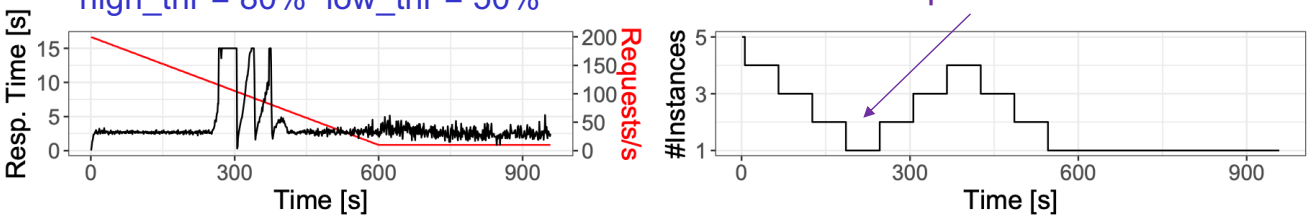
scaling period = 30 s.
high_thr = 80% low_thr = 25%

over-provisioned



scaling period = 60 s.
high_thr = 80% low_thr = 50%

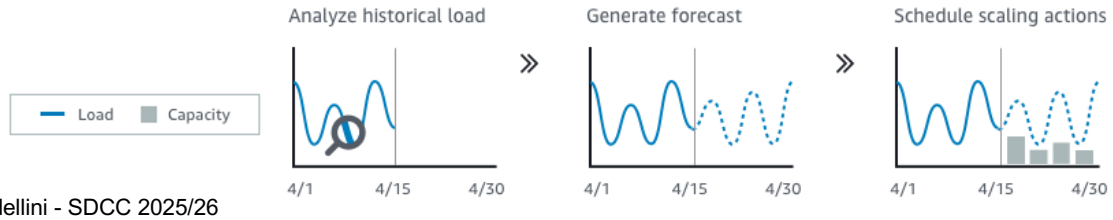
under-provisioned



Strasser et al., Autoscaler Evaluation and Configuration: A Practitioner's Guideline, *ICPE 2023*
https://research.spec.org/icpe_proceedings/2023/proceedings/p31.pdf

Example 1: Amazon EC2 Auto Scaling

- **Predictive scaling**
- ML-based policy (*proactive*)
  - Trained ML model predicts expected application traffic and EC2 usage, including daily and weekly usage patterns
    - Scaling metrics: CPU utilization, ALB request count
  - Requires historical data collected from CloudWatch
    - Minimum 1 day of data to start making predictions
    - Hourly forecasts for the next 48 hours based on the past 14 days
    - Forecast updated every 6 hours
  - Scaling action: scale out to reach a target utilization
  - Assumption: Auto Scaling group is homogeneous and all instances have equal capacity



Predictive scaling: pros and cons

✓ Proactive

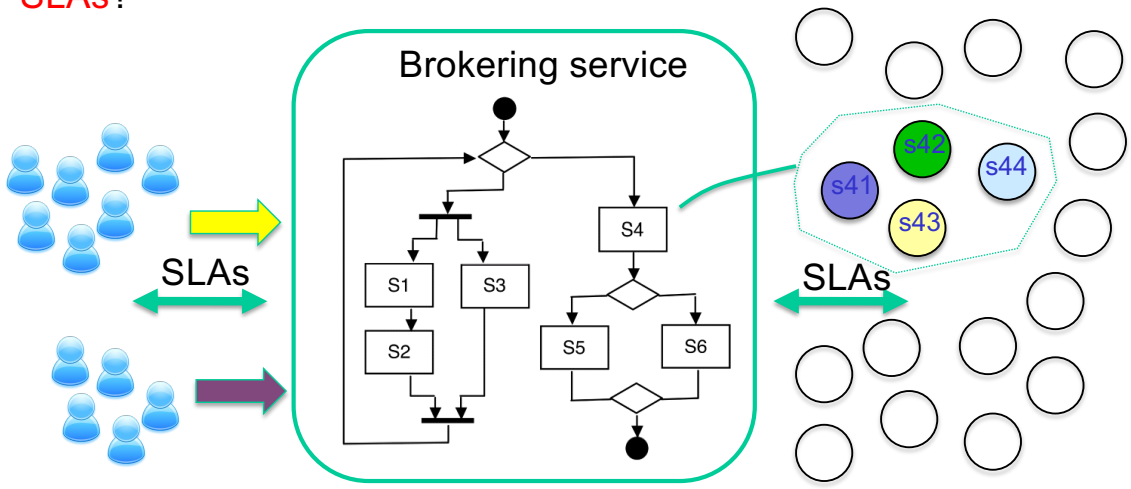
✗ Requires training
- More historical data → better forecast accuracy

✗ Choice of scaling metric (e.g., CPU utilization, request count) is critical

✗ Supports only scale-out decisions
- To scale-in a separate policy is needed

Example 2: Self-adaptive service selection

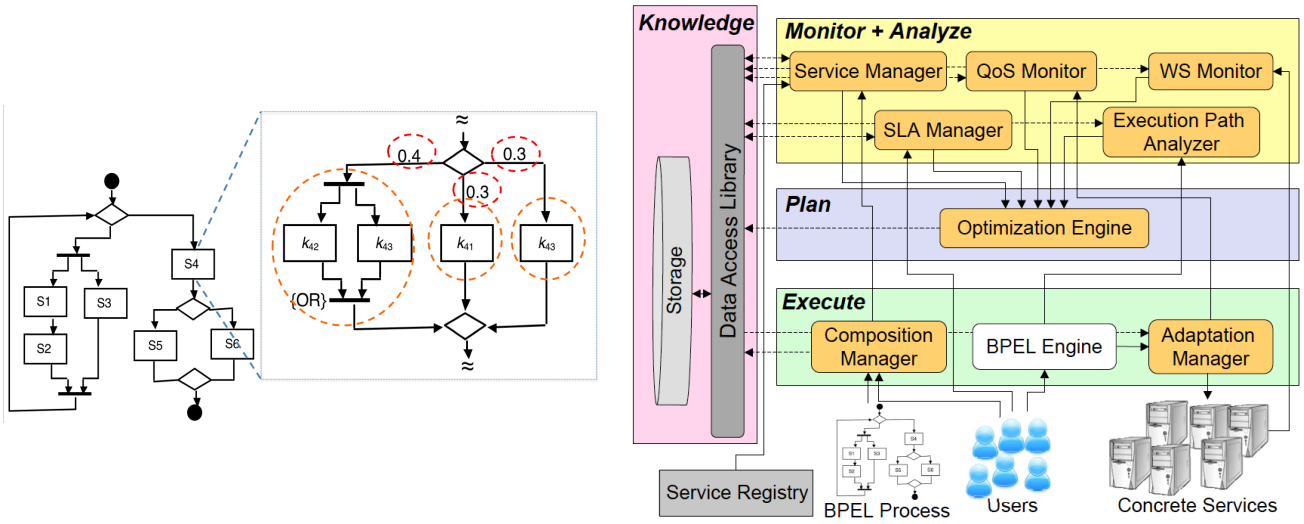
- QoS-driven self-adaptation of SOA applications
  - Each abstract service may have multiple concrete implementations with different non-functional properties
  - Which concrete services should be selected to meet application SLAs?



Cardellini et al., MOSES: a framework for QoS driven runtime adaptation of service-oriented systems, *IEEE Trans. Soft. Eng.*, 2012 http://www.ce.uniroma2.it/publications/tse2012.pdf

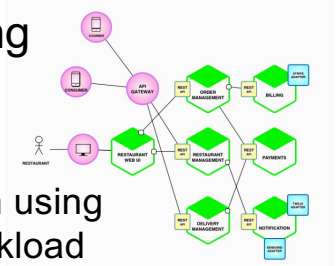
Example 2: Self-adaptive service selection

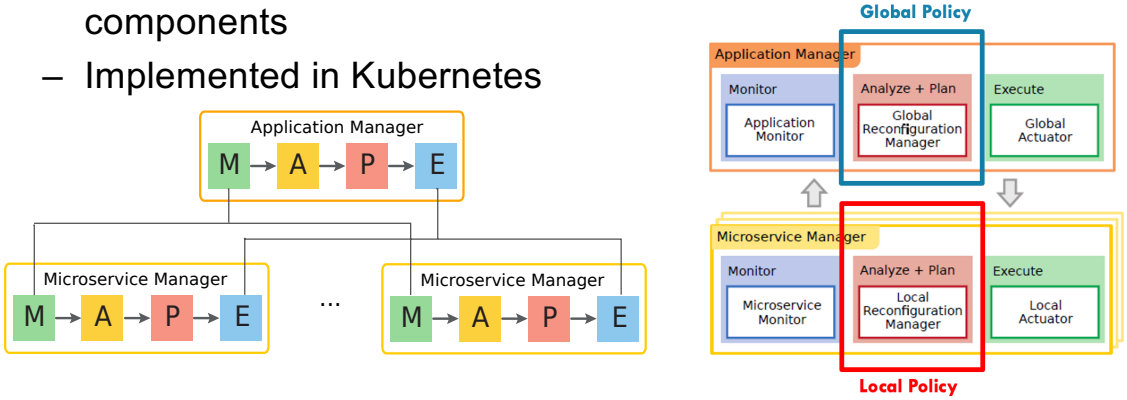
- MA**P**E Plan
  - Centralized policy: find the *optimal set of concrete services* (and their *coordination*) using linear programming optimization



Example 3: Hierarchical auto-scaling of microservices

- Hierarchical control pattern for elastic scaling of microservices-based application
  - Goal: keep app response time below maximum
  - Each Microservice Mngr. determines scale-out/in using a local policy based on queueing theory and workload prediction and sends scaling proposal to Application Mngr.
  - Application Mngr. coordinates proposals by approving or rejecting them and sends decisions to local Execute components
  - Implemented in Kubernetes





Rossi et al., Hierarchical scaling of microservices in Kubernetes, *Proc. IEEE ACSOS 2020*
http://www.ce.uniroma2.it/publications/acsos2020.pdf

References

- The vision of autonomic computing https://www.research.ibm.com/autonomic/research/papers/AC_Vision_Computer_Jan_2003.pdf
- An introduction to self-adaptive systems: A contemporary software engineering perspective, chapter 1 https://introsas.cs.kuleuven.be/2020ExcerptBook.pdf
- On patterns for decentralized control in self-adaptive systems https://www.ics.uci.edu/~seal/publications/2012aSefSAS.pdf